Technical task:
IoT Data Pipeline



Developped By Ahmed CHERIF

Note: To be honest, I was responsible for analyzing the data and defining the system architecture, scheduling strategies, and compression methods. However, AI assisted with the code development because I think it really saves a lot of time and apply the guidelines better then a junior developper. I guided the process to ensure that the resulting code remained maintainable and used technologies that I had already worked with in previous projects.

Task requirements summary

01

SIMULATED SENSORS

3 sources with different behaviour: slow, fast/noisy, irregular event-driven

02

SCHEDULING

Read and processed by scheduled tasks, with a real RTOS, not one ad-hoc loop

03

COMPRESSION NODE

own simple design, justified by how each sensor's data behaves

04

LOCAL DASHBOARD

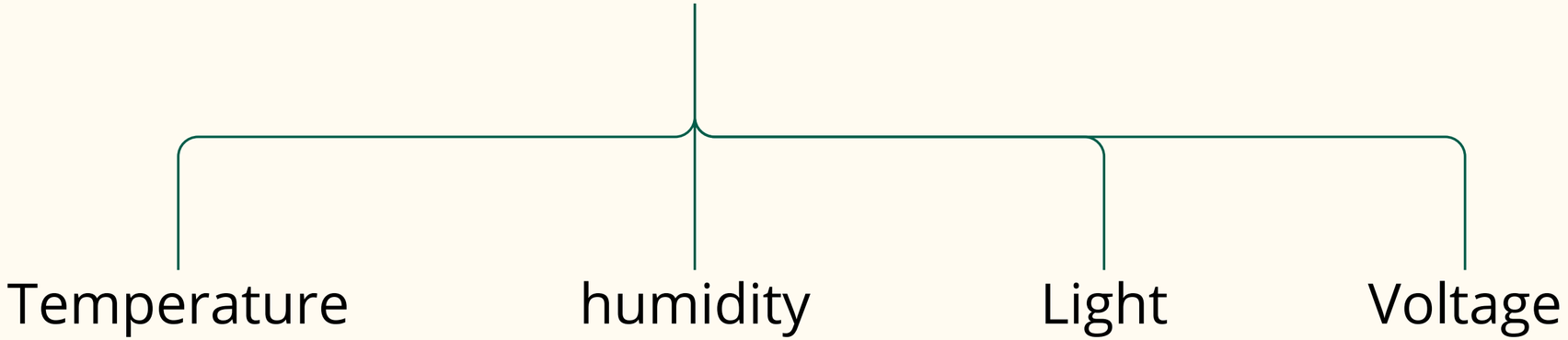
raw + compressed values, size before/after, timing and health of the system

Dataset Exploration

Intel Berkeley Data

54 Sensor nodes

4 Data sources



Dataset Exploration

From the source txt file when went to 54 csv files, every file represents a node and we added header.

Made several measurments across the data nodes and also leaned on another github repo that help the analysis.

Mote number 7 is the one that will be used across this project but all the conclusions can be applied to other motes data.

Found out that at a certain point when the voltage gets down the values become some kind of hallucinations so we cleaned the data a little bit.

Dataset Exploration

The most relevant measurment for me is delta which is the difference the value at t=n+1 at the previous value at t=n

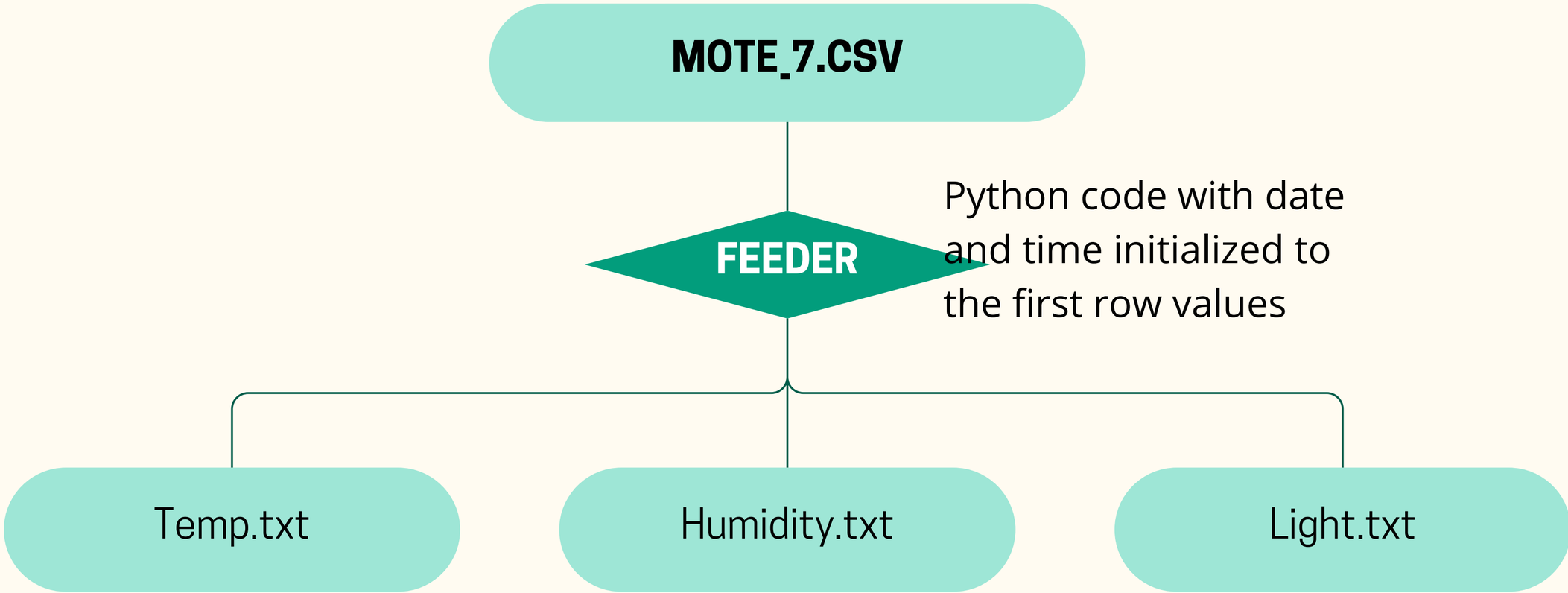
date	time	epoch	moteid	temperature	humidity	light	voltage	delta_temp	steps_temp	absolute_steps_temp	delta_humidity	steps_humidity	absolute_steps_humidity	delta_light	steps_light	absolute_steps_light
28/02/2004	05:16.6	15	7	19.4298	38.9401	108.56	2.68742	0	0	0	0	0	0	0	0	0
28/02/2004	05:46.5	16	7	19.42	39.0763	104.88	2.67532	-0.0098	-1	1	0.1362	4.005882353	4.005882353	-3.68	-8	8
28/02/2004	06:16.8	17	7	19.4298	39.1443	104.88	2.68742	0.0098	1	1	0.068	2	2	0	0	0
28/02/2004	06:46.6	18	7	19.42	39.2123	104.88	2.68742	-0.0098	-1	1	0.068	2	2	0	0	0
28/02/2004	07:16.5	19	7	19.42	39.2803	104.88	2.68742	0	0	0	0.068	2	2	0	0	0
28/02/2004	07:46.5	20	7	19.4102	39.1443	104.88	2.68742	-0.0098	-1	1	-0.136	-4	4	0	0	0
28/02/2004	08:16.0	21	7	19.42	39.1443	104.88	2.68742	0.0098	1	1	0	0	0	0	0	0
28/02/2004	08:46.6	22	7	19.4004	39.3483	104.88	2.68742	-0.0196	-2	2	0.204	6	6	0	0	0
28/02/2004	09:16.5	23	7	19.4004	39.3483	104.88	2.68742	0	0	0	0	0	0	0	0	0
28/02/2004	10:16.6	25	7	19.3906	39.2463	104.88	2.68742	-0.0098	-1	1	-0.102	-3	3	0	0	0
28/02/2004	10:46.4	26	7	19.4004	39.2463	104.88	2.68742	0.0098	1	1	0	0	0	0	0	0
28/02/2004	11:16.1	27	7	19.3808	39.1443	104.88	2.68742	-0.0196	-2	2	-0.102	-3	3	0	0	0
28/02/2004	11:46.8	28	7	19.3906	39.2123	104.88	2.68742	0.0098	1	1	0.068	2	2	0	0	0
28/02/2004	12:16.2	29	7	19.3808	39.1443	104.88	2.68742	-0.0098	-1	1	-0.068	-2	2	0	0	0
28/02/2004	12:46.1	30	7	19.3906	39.1443	104.88	2.68742	0.0098	1	1	0	0	0	0	0	0
28/02/2004	13:46.2	32	7	19.3612	39.0763	104.88	2.68742	-0.0294	-3	3	-0.068	-2	2	0	0	0
28/02/2004	14:16.8	33	7	19.3612	39.0422	104.88	2.68742	0	0	0	-0.0341	-1.002941176	1.002941176	0	0	0

from this we deducted that $\Delta Temperature = s_t \times 0.0098$ $\Delta Humidity = s_h \times (0.34 + r_h)$ and $\Delta light = s_l \times 0.46$

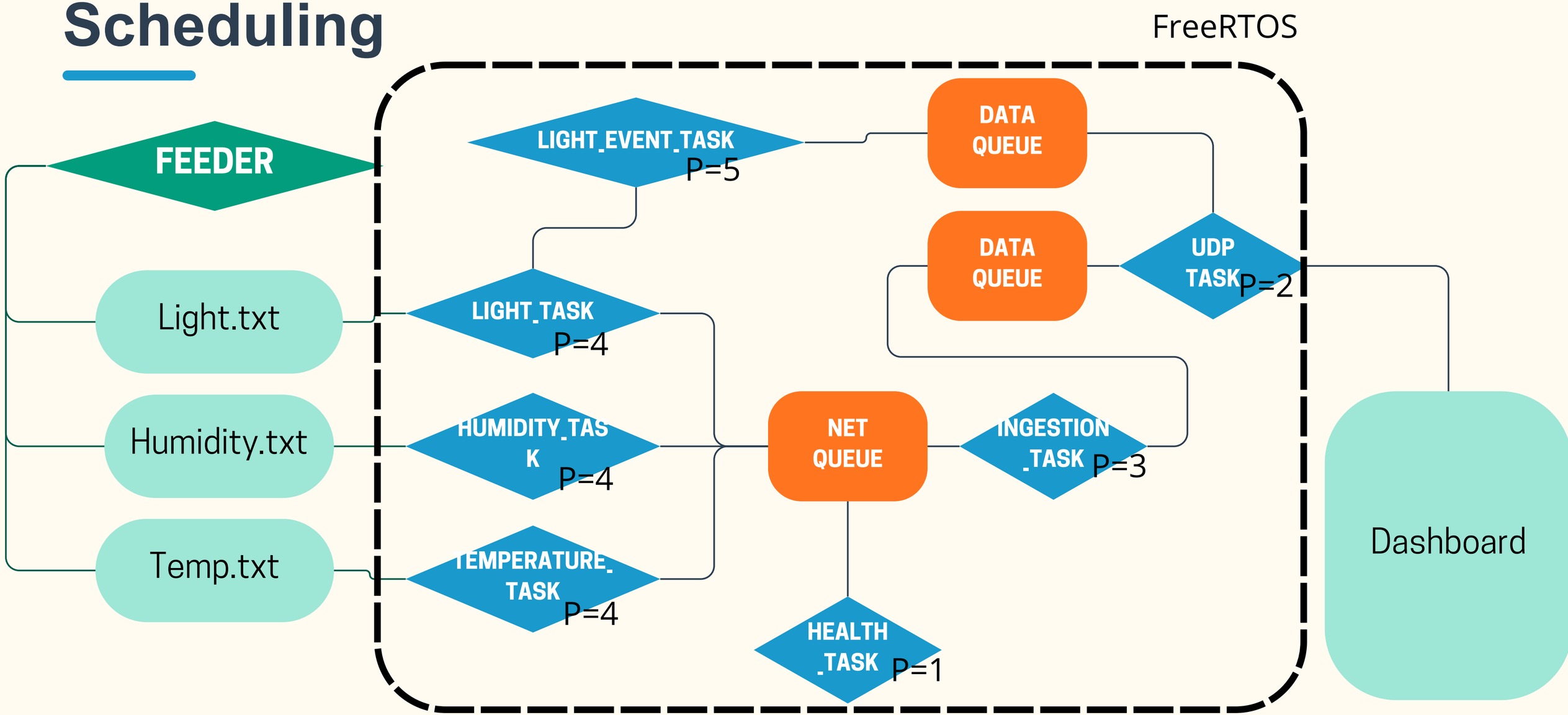
So in the compression step we just have to compress these

Sensors simulation

we said we will rely on the csv fiiles and specifically mote _7 data



Scheduling


FreeRTOS
FEEDER
Light.txt
Humidity.txt
Temp.txt
LIGHT_EVENT_TASK
P=5
LIGHT_TASK
P=4
HUMIDITY_TASK
P=4
TEMPERATURE_TASK
P=4
DATA QUEUE
DATA QUEUE
UDP TASK
P=2
NET QUEUE
INGESTION_TASK
P=3
HEALTH_TASK
P=1
Dashboard

Data compression

Following the conclusion cited in the slide 5, our datagram is composed as follows



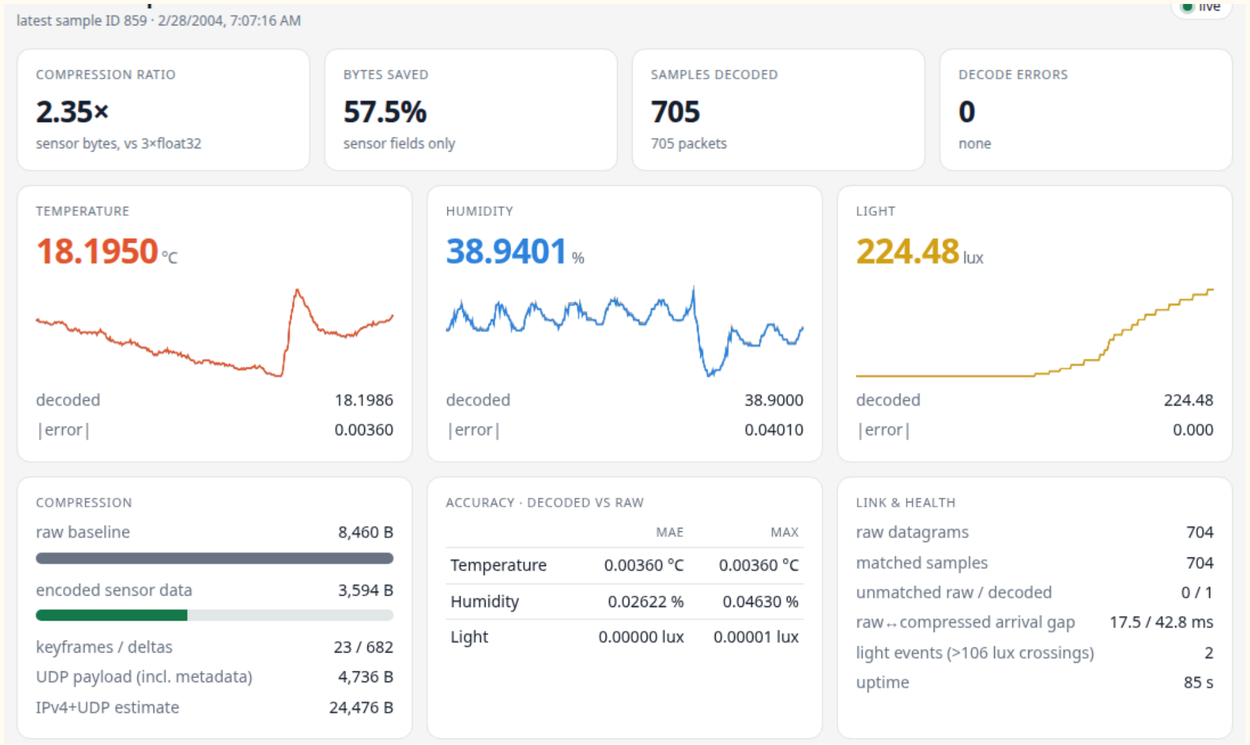
The number of bits is determined by looking at the whole data, determining the maximum needed number of bits. However I olso added one extra bit for safety

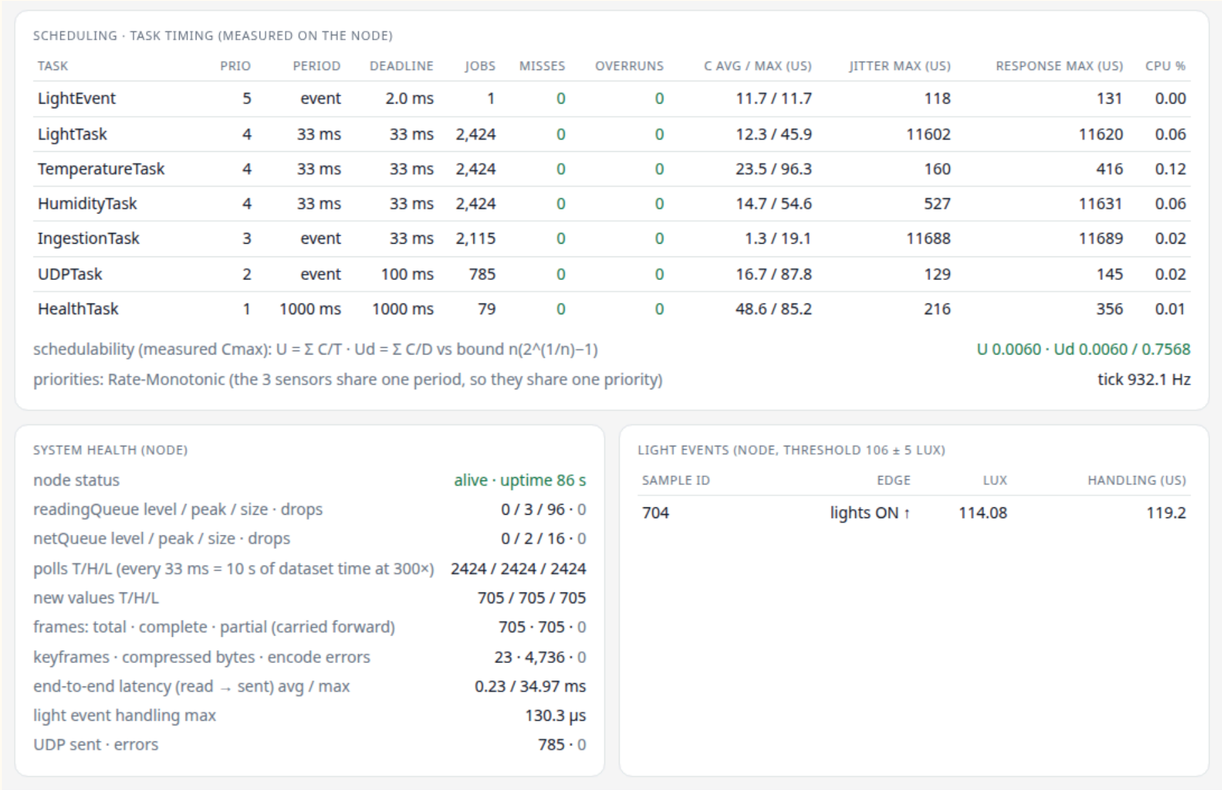
This is not optimal because we are fixing the number of bits, if if delta is 0 the declared number of bits is allocated

I tried to send seperate packet for every data type but also it did not give better results than this approach

Considering that without compression, we send 3 x 32 bits packet which is equal to 12 bytes, we went down to 5 bytes using this approach (ignoring the id byte)

Dashboard





Via the dashboard we can see:

- raw vs decoded values per sensor,
- compression ratio, bytes saved, keyframes/deltas
- accuracy and unmatched rows

- per-task timing, misses, overruns, CPU, schedulability
- queues, frames, latencies, light events

Repo Link
https://github.com/AhmedCh24/data_compression_bitteiler_technical_task